Akka Classic — Fast Exam Reference & Code Playbook (NSDS)

1. Exam Decision Matrix — "Which Pattern Do I Need?"

Exam Requirement	Akka Mechanism to Use	Jump to
Filter messages by type vs. by field value	`.match(Class, handler)` vs. `.match(Class, msg -> cond, handler)`	Pattern 1
Pass `ActorRef` or config parameters to an actor constructor	`Props.create(MyActor.class, () -> new MyActor(arg1, arg2))`	Pattern 1
Actor changes how it reacts to messages based on its mode/state	`getContext().become(nextBehavior())` (Finite State Machine)	Pattern 2
Postpone messages when a precondition fails (e.g., `counter == 0` or uninitialized)	Extend `AbstractActorWithStash` → `stash()` and `unstashAll()`	Pattern 3
Wait for a reply **without blocking** the actor thread	`AbstractActorWithStash` + `become(waiting())` + `.matchAny(m -> stash())`	Pattern 4A
Wait for a reply **synchronously** (blocking)	`Patterns.ask(target, msg, timeoutMs)` + `Await.result(future, duration)`	Pattern 4B
Forward message through a router/proxy keeping original sender	`target.tell(msg, sender())` (or `target.forward(msg, getContext())`)	Section 2
Handle actor crashes (`resume`, `restart`, `stop`, `escalate`)	Parent overrides `supervisorStrategy()` with `OneForOneStrategy` / `AllForOneStrategy`	Pattern 5
Protect critical state while running a fault-prone computation	Keep critical state in **Parent Supervisor**, delegate risky task to **Child Worker**	Pattern 5B
Connect Client and Server across separate processes/machines	`ConfigFactory.parseFile` (`remote.conf`) + `getContext().actorSelection("akka.tcp://...")`	Pattern 6

2. Essential Imports & One-Liner API Cheat Sheet

```
// --- IMPORTS ---
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import akka.pattern.Patterns;
import com.typesafe.config.Config;
import com.typesafe.config.ConfigFactory;
import scala.concurrent.Await;
import scala.concurrent.Future;
import java.io.File;
import java.io.Serializable;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.*;
import static akka.pattern.Patterns.ask;
import static java.util.concurrent.TimeUnit.SECONDS;

// --- SYSTEM & ACTOR CREATION ---
ActorSystem sys = ActorSystem.create("System");
ActorRef actor = sys.actorOf(MyActor.props(arg), "myActor"); // Top-level actor
ActorRef child = getContext().actorOf(ChildActor.props(), "child"); // Supervised child actor

// --- SENDING, REPLYING & FORWARDING ---
actor.tell(new Msg(), ActorRef.noSender()); // From main() (no sender)
```

```
actor.tell(new Msg(), self());              // From inside an actor
sender().tell(new ReplyMsg(), self());      // Reply to whoever sent current message
target.tell(msg, sender());                 // Forward preserving original sender()

// --- STOPPING ---
getContext().stop(child); // Stop a child actor
getContext().stop(self()); // Stop self
sys.terminate();          // Shut down entire ActorSystem

// --- SYNCHRONOUS ASK (TWO EQUIVALENT FORMS) ---
// Form 1: Patterns.ask + Await.result
Future<Object> f = Patterns.ask(server, new RequestMsg(), 5000);
ReplyMsg rep = (ReplyMsg) Await.result(f, scala.concurrent.duration.Duration.create(5, SECONDS));

// Form 2: ask + future.result (often used to get a child ActorRef from a Supervisor)
scala.concurrent.duration.Duration timeout = scala.concurrent.duration.Duration.create(5, SECONDS);
Future<Object> fChild = ask(supervisor, Props.create(WorkerActor.class), 5000);
ActorRef worker = (ActorRef) fChild.result(timeout, null);
```

3. Core Theory Rules (Rapid Review for Open/Multiple-Choice Questions)

3.1 Concurrency, Atomicity & Mailbox Semantics

• **3 Actor Actions on Message Receive**: (1) local decisions (mutate private state or `become()`), (2) send *0, 1,* or more messages, (3) spawn child actors.

• **Why Locks Are Never Needed**: Actor state is private. Messages are processed **one at a time atomically** (*as if inside a* `synchronized` *method of a class where all methods are* `synchronized`). Concurrent arrivals wait in the mailbox.

• **Pattern Matching Order**: `createReceive()` evaluates `.match( ... )` clauses **top-to-bottom** and runs the **first match**. If **no clause matches**, the message is **silently discarded**.

• **Ordering Guarantees**: Default mailbox is **FIFO**.

  • **Single sender → receiver**: **FIFO order** guaranteed (over local memory or TCP).

  • **Multiple senders → receiver**: arrival interleaving is **non-deterministic**.

• **Immutability**: Messages **must be immutable** (`private final` fields, defensive copies for collections). The framework **cannot check this statically** at compile time, but mutable messages cause data races.

3.2 Lifecycle, Addressing & Communication

• `ActorPath` **vs.** `ActorRef` **vs.** `ActorSelection`:

  • `ActorPath`: logical hierarchical name. Freed and reusable after the actor stops.

  • `ActorRef`: points to a specific **incarnation** (`ActorPath` + random `UID`). **Remains valid across restarts** and can be sent inside messages.

  • `ActorSelection`: path query (supports wildcards); resolves the underlying `ActorRef` via `Identify` → `ActorIdentity`.

  • **Many-to-Many**: 1 address can represent multiple actors (load-balancing/routing); 1 actor can have multiple addresses.

• **Lifecycle Hooks**:

  • `preStart()`: called right after creation, before first message.

  • `preRestart(reason, msg)`: called on the **old** crashing instance before restart.

  • `postRestart(reason)`: called on the **new** instance after restart.

  • `postStop()`: called when permanently stopped (`stop()`, `PoisonPill`, `sys.terminate()`); sends `Terminated` to watchers.

• **Best-Effort Communication ("Less is more")**:

  • Direct point-to-point (no mediating channel). **At-most-once** (no duplicates, messages can be lost) and no global order.

  • *Why?* IP is best-effort, TCP cannot mask network partitions, and stronger guarantees are expensive/not always needed (can be built at application level).

3.3 Stash, Fault Tolerance & Clustering

• **Stash (`AbstractActorWithStash`)**:

  - `stash()` buffers the current message. `unstashAll()` extracts all stashed messages **in FIFO order** and **prepends them to the front of the mailbox** (processed before newer mailbox messages).
  - *Ex 2 Trace (`D, D, I, D, I, I` from `0`)*: `D1` stashed → `D2` stashed → `I1` (`1`, unstashes `D1, D2`) → `D1` (`0`) → `D2` re-stashed → `D3` stashed → `I2` (`1`, unstashes `D2, D3`) → `D2` (`0`) → `D3` re-stashed → `I3` (`1`, unstashes `D3`) → `D3` (`0`). State evolution: `1, 0, 1, 0, 1, 0`.
- **Supervision & Restart Rules**:
  - Every actor is supervised by its **parent** (`getContext().actorOf()`).
  - **Hierarchy of Responsibility**: keep critical data in the **supervisor**; delegate dangerous subtasks to **children**. A supervisor strategy depends only on exception type and error count (not which child failed), so supervise homogeneous children (build deeper trees for different roles).
  - **On Restart**: (1) `ActorRef` **(address) does NOT change**, (2) **Mailbox is preserved** (lives outside actor memory), (3) **Internal state is reset** to initial constructor values.
- **Strategies & Directives**:
  - `OneForOneStrategy` (only crashed child) vs. `AllForOneStrategy` (crashed child + all siblings).
  - `resume()` (keep instance & state), `restart()` (new instance, reset state), `stop()` (terminate child), `escalate()` (propagate to parent).
  - If faults exceed `maxNrOfRetries` within `withinTimeRange`, the child is **stopped**.
- **Clustering**:
  - **Node**: `hostname:port:uid` (process/`ActorSystem`); membership is decoupled from hosted actors.
  - **Protocol**: Decentralized P2P overlay using **gossip** with **vector clocks** (partial event ordering) + automatic failure detection.
  - **Gossip Convergence**: reached when a node proves its observed state matches all other nodes. **Blocked while any node is** `"unreachable"` (must become reachable or be removed).Managed by the **Leader** role (not a SPOF).
  - **Seed Nodes**: joining node contacts **all seed nodes** (at least one active) and sends `join` to the **first seed node that replies**.
  - **Cluster Tools**: *Cluster Singleton* (1 instance in cluster), *Cluster Sharding* (distribute by logical ID), *Distributed Data* (key-value CRDT store), *Cluster Metrics* (health/load monitoring).

---

4. Complete Commented Code Patterns

Pattern 1 — Immutable Messages, Basic Actor & Predicates

(Ref: `TemplateMessage.java`, `TemplateActor.java`, `MainTemplate.java`)

```
// 1. IMMUTABLE MESSAGE (final fields, defensive copy for collections, Serializable)
public class OpMsg implements Serializable {
    private static final long serialVersionUID = 1L;
    public enum Type { INC, DEC }

    private final Type type;
    private final int amount;
    private final List<String> tags;

    public OpMsg(Type type, int amount, List<String> tags) {
        this.type = type;
        this.amount = amount;
        // Defensive copy so sender cannot mutate the list after sending
        this.tags = Collections.unmodifiableList(new ArrayList<>(tags));
    }

    public Type getType() { return type; }
    public int getAmount() { return amount; }
    public List<String> getTags() { return tags; }
}

// 2. STATEFUL ACTOR WITH PREDICATE & CLASS MATCHING
public class CounterActor extends AbstractActor {
    private int counter; // Private state: thread-safe without locks!
```

```
    public CounterActor(int initialValue) {
        this.counter = initialValue;
    }

    @Override
    public Receive createReceive() {
        // Clauses evaluated top-to-bottom: put specific predicates BEFORE general class matches!
        return receiveBuilder()
                // Match by class + predicate selector
                .match(OpMsg.class, m -> m.getType() == OpMsg.Type.INC && m.getAmount() > 0, this::onInc)
                .match(OpMsg.class, m -> m.getType() == OpMsg.Type.DEC && m.getAmount() > 0, this::onDec)
                // Match by class only
                .match(String.class, cmd -> {
                    // Reply to sender with current state
                    sender().tell(counter, self());
                })
                .build();
    }

    private void onInc(OpMsg msg) { counter += msg.getAmount(); }
    private void onDec(OpMsg msg) { counter -= msg.getAmount(); }

    // Factory method with constructor argument
    public static Props props(int initialValue) {
        return Props.create(CounterActor.class, () -> new CounterActor(initialValue));
    }
}
```

Pattern 2 — Dynamic Behavior / State Machine (`become`)

(Ref: `StateMachineActorTemplate.java`)

```
public class AlarmActor extends AbstractActor {
    public static class ActivateMsg implements Serializable {}
    public static class DeactivateMsg implements Serializable {}
    public static class TriggerMsg implements Serializable {}

    @Override
    public Receive createReceive() {
        return disabled(); // Initial behavior state
    }

    // State 1: DISABLED (TriggerMsg is not matched -> automatically discarded!)
    private Receive disabled() {
        return receiveBuilder()
                .match(ActivateMsg.class, msg -> {
                    System.out.println("Switched to ENABLED");
                    getContext().become(enabled()); // Transition to enabled()
                })
                .build();
    }

    // State 2: ENABLED
    private Receive enabled() {
        return receiveBuilder()
                .match(TriggerMsg.class, msg -> System.out.println("ALARM!"))
                .match(DeactivateMsg.class, msg -> {
                    System.out.println("Switched to DISABLED");
                    getContext().become(disabled()); // Transition to disabled()
                })
                .build();
    }

    public static Props props() { return Props.create(AlarmActor.class); }
}
```

Pattern 3 — Conditional Processing with `stash()` & `unstashAll()`

(Ref: `StashActorTemplate.java` & Lab 1 `ex2`)

```
// MUST extend AbstractActorWithStash (NOT AbstractActor)
public class NonNegativeCounterActor extends AbstractActorWithStash {
    public static class IncMsg implements Serializable {}
    public static class DecMsg implements Serializable {}

    private int counter = 0;

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(IncMsg.class, this::onInc)
                .match(DecMsg.class, this::onDec)
                .build();
    }

    private void onInc(IncMsg msg) {
        ++counter;
        System.out.println("Counter = " + counter);
        // State changed → prepend all stashed DecMsg back to the front of mailbox in FIFO order!
        unstashAll();
    }

    private void onDec(DecMsg msg) {
        if (counter > 0) {
            --counter;
            System.out.println("Counter = " + counter);
        } else {
            // Precondition failed (counter == 0) → postpone message in stash buffer
            stash();
        }
    }

    public static Props props() { return Props.create(NonNegativeCounterActor.class); }
}
```

Pattern 4 — Client-Server Request-Reply: Async (`stash`) vs. Sync (`ask`)

(Ref: `RequestReplyTemplate.java` & Lab 1 `ex3`)

```
// --- SERVER ACTOR (Works identically with both AsyncClient and AskClient) ---
public class ServerActor extends AbstractActor {
    private final Map<String, String> db = new HashMap<>();

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(PutMsg.class, m → db.put(m.getKey(), m.getVal()))
                .match(GetMsg.class, m → {
                    // Always reply to sender() passing self()
                    sender().tell(new ReplyMsg(m.getKey(), db.get(m.getKey())), self());
                })
                .build();
    }
    public static Props props() { return Props.create(ServerActor.class); }
}

// --- 4A. NON-BLOCKING CLIENT (become + stash + unstashAll) ---
public class AsyncClientActor extends AbstractActorWithStash {
    private final ActorRef server;
    public AsyncClientActor(ActorRef server) { this.server = server; }

    @Override
    public Receive createReceive() { return idle(); }

    private Receive idle() {
```

```
        return receiveBuilder()
                .match(PutMsg.class, m -> server.tell(m, self()))
                .match(GetMsg.class, m -> {
                    server.tell(m, self());                  // 1. Send async request
                    getContext().become(waitingForReply());  // 2. Switch to waiting state
                })
                .build();
    }

    private Receive waitingForReply() {
        return receiveBuilder()
                .match(ReplyMsg.class, reply -> {
                    System.out.println("Got reply: " + reply.getVal());
                    unstashAll();                            // 3. Restore postponed messages
                    getContext().become(idle());             // 4. Return to normal behavior
                })
                .matchAny(msg -> stash())                    // Postpone any other message while waiting
                .build();
    }

    public static Props props(ActorRef s) { return Props.create(AsyncClientActor.class, () -> new
AsyncClientActor(s)); }
}

// --- 4B. SYNCHRONOUS BLOCKING CLIENT (Patterns.ask + Await.result) ---
public class AskClientActor extends AbstractActor {
    private static final long TIMEOUT_MS = 5000;
    private final ActorRef server;
    public AskClientActor(ActorRef server) { this.server = server; }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(PutMsg.class, m -> server.tell(m, self()))
                .match(GetMsg.class, m -> {
                    // Blocks current thread until ReplyMsg arrives or 5s timeout expires
                    Future<Object> future = Patterns.ask(server, m, TIMEOUT_MS);
                    ReplyMsg reply = (ReplyMsg) Await.result(
                            future, scala.concurrent.duration.Duration.create(5, SECONDS));
                    System.out.println("Got sync reply: " + reply.getVal());
                })
                .build();
    }

    public static Props props(ActorRef s) { return Props.create(AskClientActor.class, () -> new
AskClientActor(s)); }
}
```

Pattern 5 — Fault Tolerance & Supervision Hierarchy

(Ref: `SupervisionTemplate.java`)

```
// --- 5A. SUPERVISOR & SUPERVISED CHILD (Slides 11-13) ---
public class CounterSupervisorActor extends AbstractActor {
    // Up to 10 faults within 1 minute (if exceeded -> child is stopped)
    private static final SupervisorStrategy strategy =
        new OneForOneStrategy( // Use AllForOneStrategy to apply directive to ALL children
            10,
            Duration.ofMinutes(1),
            DeciderBuilder
                .match(ArithmeticException.class, e -> SupervisorStrategy.resume())    // Keep state
                .match(NullPointerException.class, e -> SupervisorStrategy.restart())  // Reset state
                .match(IllegalArgumentException.class, e -> SupervisorStrategy.stop()) // Kill child
                .match(Exception.class, e -> SupervisorStrategy.restart())
                .matchAny(o -> SupervisorStrategy.escalate())                          // Escalate to parent
                .build());

    @Override
    public SupervisorStrategy supervisorStrategy() { return strategy; }
```

```
    @Override
    public Receive createReceive() {
        // Spawns child inside supervisor's context and returns child ActorRef to sender
        return receiveBuilder()
                .match(Props.class, props -> sender().tell(getContext().actorOf(props), self()))
                .build();
    }
    public static Props props() { return Props.create(CounterSupervisorActor.class); }
}

public class FaultyChildActor extends AbstractActor {
    private int state = 0;

    @Override
    public Receive createReceive() {
        return receiveBuilder().match(Integer.class, this::onCode).build();
    }

    // Must declare 'throws Exception' if throwing checked exceptions!
    void onCode(Integer code) throws Exception {
        if (code == 0) {
            ++state;
            System.out.println("State = " + state);
        } else if (code == -1) {
            throw new Exception("Crash! Triggers restart() -> state resets to 0");
        } else if (code == -2) {
            throw new ArithmeticException("Minor error! Triggers resume() -> state kept");
        }
    }

    @Override
    public void preRestart(Throwable reason, Optional<Object> msg) { System.out.print("Restarting ... "); }
    @Override
    public void postRestart(Throwable reason) { System.out.println("done! State is " + state); }

    public static Props props() { return Props.create(FaultyChildActor.class); }
}

// --- MAIN: SPAWNING CHILD THROUGH SUPERVISOR ---
// Future<Object> f = ask(supervisor, Props.create(FaultyChildActor.class), 5000);
// ActorRef child = (ActorRef) f.result(scala.concurrent.duration.Duration.create(5, SECONDS), null);
```

Pattern 5B — Hierarchy of Responsibility (Safe Parent + Risky Child)

```
// Parent holds critical state that must NEVER be lost; child executes dangerous operations
public class SafeParentSupervisor extends AbstractActor {
    private final Map<String, Integer> criticalData = new HashMap<>(); // Safe across child crashes!
    private ActorRef worker;

    private static final SupervisorStrategy strategy = new OneForOneStrategy(
            10, Duration.ofMinutes(1),
            DeciderBuilder.match(Exception.class, e -> SupervisorStrategy.restart()).build());

    @Override
    public SupervisorStrategy supervisorStrategy() { return strategy; }

    @Override
    public void preStart() {
        worker = getContext().actorOf(RiskyWorker.props(), "worker");
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(TaskMsg.class, task -> worker.tell(task, self())) // Delegate risky task
                .match(ResultMsg.class, res -> criticalData.put(res.getId(), res.getValue()))
                .build();
    }
}
```

Pattern 6 — Remote Distribution over TCP

(Ref: `RemoteDistributionTemplate.java` *&* `remote.conf` *)*

```
# 1. File "remote.conf"
akka {
  actor { provider = remote }
  remote {
    enabled-transports = ["akka.remote.netty.tcp"]
    netty.tcp {
      hostname = "127.0.0.1"
      port = 6123
    }
  }
}
```

```
// 2. Server Main: start remote ActorSystem listening on 127.0.0.1:6123
Config conf = ConfigFactory.parseFile(new File("remote.conf"));
ActorSystem sys = ActorSystem.create("Server", conf);
sys.actorOf(ServerActor.props(), "serverActor");

// 3. Client Actor: lookup remote server by address using ActorSelection
public class RemoteClientActor extends AbstractActor {
    // Format: akka.tcp://<ActorSystemName>@<Hostname>:<Port>/user/<ActorName>
    private final ActorSelection server =
            getContext().actorSelection("akka.tcp://Server@127.0.0.1:6123/user/serverActor");

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(GetMsg.class, msg -> server.tell(msg, self())) // Identical to local tell()!
                .match(ReplyMsg.class, rep -> System.out.println("Remote reply: " + rep.getVal()))
                .build();
    }
}
```